تمرین

۰۲

گرافیک کامپیوتری

COMPUTER GRAPHICS

Rasterization, Symmetry, Clipping, and Filling

نیمسال اول ۱۴۰۵–۱۴۰۶

تاریخ بارگذاری تمرین	جمعه، ۲۴ مهر ۱۴۰۵	مهلت تحویل	جمعه، ۱ آبان ۱۴۰۵، ۲۳:۵۹
مدرس	دکتر فاطمه پناهی	تدریس‌یار	مجید نامی
نمره تمرین	۱۰۰ نمره + ۱۰ امتیازی	محل تحویل	سامانه کورسز
زبان برنامه‌نویسی	Python ۳٫۱۰ یا بالاتر	نوع تمرین	فردی

۱ هدف تمرین

در این تمرین چند مبحث کلاس، یعنی Rasterization، تقارن، برش با Cohen–Sutherland و پر کردن با scan-line، را کنار هم می‌گذارید و یک صحنه‌ی زنده می‌سازید: ستاره‌ای داخل یک قاب حرکت می‌کند و می‌چرخد و فقط بخشی از آن که داخل پنجره‌ی قاب است دیده می‌شود. الگوریتم‌ها در این سند دوباره توضیح داده نشده‌اند؛ برای جزئیات به اسلایدهای Lecture ۴ (scan-line)، Lecture ۳ (Bresenham) و Lecture ۵ (Cohen–Sutherland) مراجعه کنید. فقط anti-aliasing، چون هنوز در کلاس تدریس نشده، در بخش امتیازی مرحله‌به‌مرحله توضیح داده شده است.

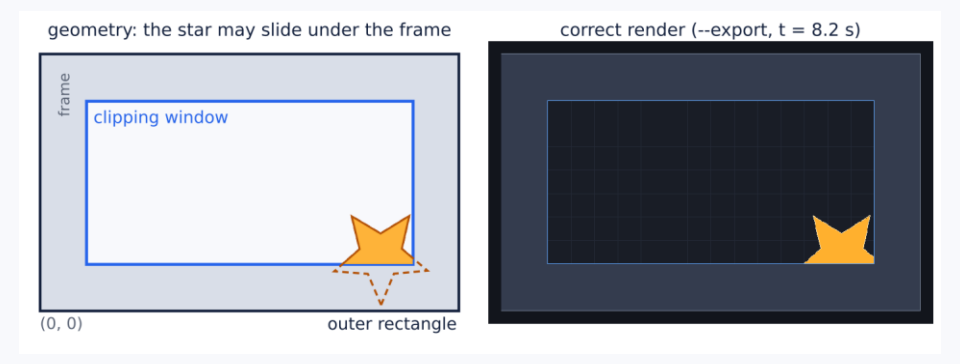
۲ شرح تمرین

ایده‌ی صحنه: قاب، پنجره و ستاره

صحنه دو مستطیل دارد. مستطیل بیرونی مرز صحنه است و مستطیل داخلی «clipping window» یا دهانه‌ی قاب. فاصله‌ی بین این دو مستطیل خودِ قاب است. ستاره در محدوده‌ی مستطیل بیرونی حرکت می‌کند و می‌چرخد و ممکن است از مرز پنجره‌ی داخلی رد شود؛ ولی فقط قسمتی از آن که داخل پنجره است باید دیده شود.

- ستاره هیچ‌وقت از مستطیل بیرونی بیرون نمی‌رود؛ حرکت صحنه طوری تنظیم شده که کل ستاره همیشه داخل این مرز بماند.
- ستاره می‌تواند به ناحیه‌ی قاب برسد؛ قسمتی از ستاره که روی قاب می‌افتد باید دیده نشود.
- قاب قبل از ستاره رسم می‌شود. پس اگر برش را درست انجام ندهید، رنگ یا لبه‌ی ستاره روی قاب می‌افتد و اشتباه همان لحظه معلوم می‌شود.

همه‌ی چیزهایی که روی صفحه می‌بینید (قاب، پنجره، شبکه و ستاره) فقط با توابعی رسم می‌شوند که خودتان می‌نویسید. صحنه (`scene.py`) و پنجره‌ی نمایش (`screensaver.py`) را ما در اختیارتان می‌گذاریم و شما فقط `rasterize.py` را کامل می‌کنید. تصویر هر frame فقط از روی زمان t ساخته می‌شود.



شکل ۱: چپ: هندسه‌ی صحنه؛ ستاره از پنجره رد شده و قسمت خط‌چین زیر قاب است. راست: نتیجه‌ی درست رندر (خروجی `--export` برای $t = 8.2$ ثانیه)؛ قسمتی از ستاره که روی قاب می‌افتد نباید نمایش داده شود.

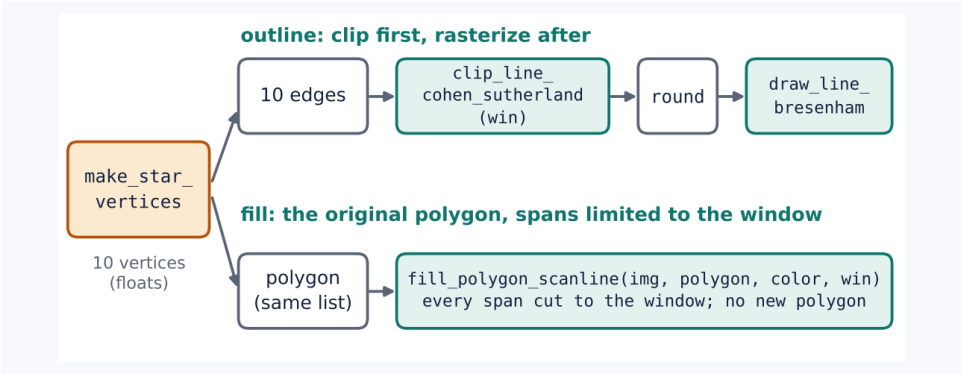
بخش	تابعی که می‌نویسید	کاربرد در صحنه
۱ · تقارن ستاره	`make_star_vertices`	ساخت ۱۰ رأس ستاره با تکرار یک الگو (تقارن پنج‌گانه)
۲ · Bresenham	`draw_line_bresenham`	رسم قاب، پنجره، شبکه و بخش دیده‌شده‌ی لبه‌های ستاره
۳ · Cohen–Sutherland	`clip_line_cohen_sutherland`	برش هر لبه‌ی ستاره با پنجره‌ی داخلی، پیش از رسم
۴ · scan-line	`fill_polygon_scanline`	پر کردن ستاره و محدود کردن هر span به پنجره
امتیازی · anti-aliasing	`fill_polygon_aa`	نرم کردن لبه‌ی ستاره با supersampling

جدول ۱: اجزای تمرین و نقش آن‌ها در خط لوله‌ی رندر

خط لوله‌ی رندر

برای ساختن هر frame، اول با استفاده از تقارن، ۱۰ رأس ستاره را به ترتیب می‌سازید. این ۱۰ رأس هندسه‌ی ستاره را مشخص می‌کنند: ۱۰ لبه‌ی ستاره از آن‌ها به دست می‌آید و خود همین فهرست هم ورودی scan-line است. از اینجا به بعد کار دو مسیر جدا دارد، یکی برای لبه‌ها و یکی برای داخل ستاره، ولی هر دو از یک پنجره استفاده می‌کنند (شکل ۲). لازم نیست چندضلعی بریده‌شده‌ی جدیدی بسازید؛ polygon clipping جزو این تمرین نیست.

در مسیر لبه‌ها، `scene.py` هر لبه را به تابع Cohen–Sutherland شما می‌دهد، دو سر پاره‌خطِ باقی‌مانده را گرد می‌کند و با Bresenham شما می‌کشد. این بخش آماده است و شما فقط توابع جدول ۱ را می‌نویسید.



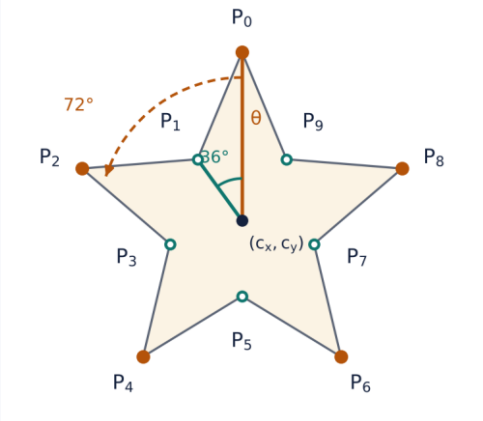
شکل ۲: لبه‌های ستاره اول با Cohen–Sutherland بریده و بعد با Bresenham رسم می‌شوند؛ داخل ستاره با scan-line پر می‌شود و هر span پیش از نوشته شدن به همان پنجره محدود می‌شود.

نکته‌ی مهم: ترتیب «اول برش، بعد رسم» فقط برای لبه‌هاست. داخل ستاره را با خودِ چندضلعی اصلی و scan-line پر کنید و هر span را به محدوده‌ی پنجره محدود کنید.

بخش ۱ · تولید رأس‌های ستاره با تقارن

تابع `make_star_vertices(cx, cy, r_outer, r_inner, theta)` باید ۱۰ رأس ستاره‌ی پنج‌پر را با تقارن دورانی پنج‌گانه بسازد و به ترتیب دور ستاره برگرداند. ده رأس را دستی ننویسید (hard-code نکنید): یک جفت رأس بیرونی و داخلی بسازید و چهار جفت دیگر را با تکرار همین الگو، هر بار ۷۲ درجه چرخیده‌تر، به دست بیاورید. خروجی فقط فهرست مرتب رأس‌هاست؛ در ادامه همین فهرست برای ساختن ۱۰ لبه، برش، Bresenham و جدول‌های ET و AET در scan-line استفاده می‌شود.

- مرکز ستاره (c_x, c_y) است. رأس ۰ بیرونی است: به فاصله‌ی `r_outer` از مرکز و در جهت `theta` (به رادیان، خلاف عقربه‌های ساعت از محور x). رأس ۱ داخلی است: به فاصله‌ی `r_inner` و در جهت `theta` به‌علاوه‌ی ۳۶ درجه. ستاره پنج sector ۷۲ درجه‌ای دارد و رأس داخلی دقیقاً وسط هر sector است.
- جفت‌های بعدی همان جفت اول‌اند که ۷۲، ۱۴۴، ۲۱۶ و ۲۸۸ درجه چرخیده‌اند. می‌توانید از `math.cos` و `math.sin` استفاده کنید؛ ماتریس دوران و مبحث Transformations لازم نیست.
- مختصات رأس‌ها اعشاری می‌ماند. گرد کردن فقط موقع فرستادن دو سر پاره‌خط به Bresenham لازم است و `scene.py` آن را انجام می‌دهد.



شکل ۳: یک جفت رأس بیرونی (پر) و داخلی (توخالی) ساخته می‌شود و همین جفت با گام ۷۲ درجه چهار بار دیگر تکرار می‌شود. در این شکل $\theta = 90°$ است.

برای امتحان، با `make_star_vertices(0, 0, 10, 4, math.pi / 2)` سه رأس اول باید این‌ها باشند (با دو رقم اعشار):

```
P0 = ( 0.00, 10.00)   P1 = (-2.35, 3.24)   P2 = (-9.51, 3.09)
```

بخش ۲ · رسم خط با Bresenham

تابع `draw_line_bresenham(img, x0, y0, x1, y1, color)` را طبق Lecture ۳ پیاده‌سازی کنید؛ شبه‌کد الگوریتم در این سند تکرار نشده است. تابع باید برای همه‌ی حالت‌ها درست کار کند: شیب مثبت و منفی، خط کم‌شیب و پرشیب، و هر ترتیبی از دو سر. هر دو پیکسل ابتدا و انتها هم باید رسم شوند.

- حلقه‌ی اصلی فقط با عدد صحیح کار کند (جمع، تفریق، ضرب در ۲ و مقایسه)؛ تقسیم و عدد اعشاری در آن ممنوع است.
- در هر قدم روی محور غالب دقیقاً یک پیکسل رسم شود و فاصله‌ی هر پیکسل تا خط ایده‌آل حداکثر نیم پیکسل باشد. اگر خط دقیقاً وسط دو پیکسل باشد، انتخاب هر کدام قبول است.
- پیکسل‌ها را با `put_pixel(img, x, y, color)` بنویسید. دو مستطیل، شبکه و لبه‌های دیده‌شده‌ی ستاره همه با همین تابع رسم می‌شوند.

برای امتحان، خط $(2, 1)$ تا $(9, 5)$ باید دقیقاً این هشت پیکسل را رنگ کند:

```
(2,1) (3,2) (4,2) (5,3) (6,3) (7,4) (8,4) (9,5)
```

بخش ۳ · برش لبه‌ها با Cohen–Sutherland

تابع `clip_line_cohen_sutherland(x0, y0, x1, y1, win)` را طبق Lecture ۵ پیاده‌سازی کنید.

- `win = (xmin, ymin, xmax, ymax)` همان مستطیل داخلی، یعنی clipping window، است.
- خروجی `(cx0, cy0, cx1, cy1)` است: قسمتی از پاره‌خط که داخل پنجره می‌افتد، با همان ترتیب دو سر ورودی. اگر هیچ قسمتی داخل پنجره نباشد، `None` برگردانید.
- مختصات y-up است، مثل کلاس: «بالا» یعنی $y > y_{\max}$ و «پایین» یعنی $y < y_{\min}$.
- مرز پنجره داخل حساب می‌شود. outcode همان `[T B R L]` اسلایدهای Lecture ۵ است؛ بیت‌ها در `rasterize.py` با نام‌های `TOP` و `BOTTOM` و `RIGHT` و `LEFT` (به ترتیب ۸، ۴، ۲ و ۱) تعریف شده‌اند.
- خروجی را گرد نکنید؛ `scene.py` تابع شما را روی هر ۱۰ لبه‌ی ستاره اجرا می‌کند و دو سرِ باقی‌مانده را خودش گرد می‌کند.

جدول ۲ نتیجه‌ی برش چهار پاره‌خط را با پنجره‌ی `(20, 30, 120, 90)` نشان می‌دهد.

پاره‌خط	outcode دو سر	نتیجه
`(-20,60)-(160,60)`	۱ و ۲	`(20,60)-(120,60)`
`(100,50)-(160,110)`	۰ و ۱۰	`(100,50)-(120,70)`
`(10,10)-(60,-20)`	۵ و ۴	`None`، چون هر دو سر زیر پنجره‌اند
`(0,40)-(30,10)`	۱ و ۴	`None`؛ بعد از یک برش هر دو سر زیر پنجره می‌افتند

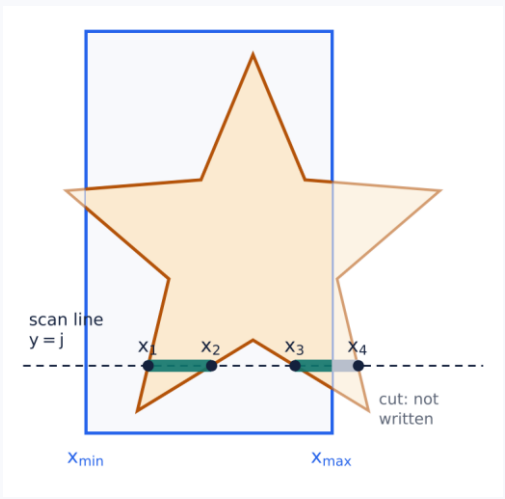
جدول ۲: برش چهار پاره‌خط با پنجره‌ی (۲۰، ۳۰، ۱۲۰، ۹۰)

راهنمایی: مثال آخر را قدم‌به‌قدم دنبال کنید. outcode دو سر ۱ و ۴ است و حاصل `&` آن‌ها صفر است، پس هنوز رد نمی‌شود. سر اول را روی ضلع چپ ببرید و ببینید outcode نقطه‌ی جدید چه می‌شود.

بخش ۴ · پر کردن ستاره با scan-line

تابع `fill_polygon_scanline(img, polygon, color, win)` ستاره‌ی مقعر را طبق Lecture ۴ با روش scan-line پر می‌کند. `polygon` همان ۱۰ رأس اصلی ستاره است؛ لازم نیست اول چندضلعی بریده‌شده‌ی جدیدی بسازید.

- منطق ET و AET، قاعده‌ی رأس‌ها، حذف لبه‌های افقی و جفت کردن تقاطع‌ها را از Lecture ۴ بردارید.
- فقط سطرهایی را پردازش کنید که در بازه‌ی عمودی پنجره‌اند، یعنی $y_{\min} \le j \le y_{\max}$.
- هر span داخل ستاره را که به دست آوردید، با بازه‌ی افقی پنجره ($x_{\min}$ تا $x_{\max}$) اشتراک بگیرید و فقط قسمت مشترک را با `fill_span(img, y, x_first, x_last, color)` بنویسید.
- تابع باید برای چندضلعی مقعر و برای هر دو جهت رأس‌ها (ساعتگرد یا پادساعتگرد) درست کار کند. هم باید روی تصویر رنگی $(H, W, 3)$ کار کند و هم روی mask دوبعدی (H, W)؛ بخش امتیازی از mask استفاده می‌کند.



شکل ۴: یک سطر ستاره‌ی مقعر را در چهار نقطه‌ی x_1 تا x_4 قطع می‌کند. هر span داخل ستاره با بازه‌ی $[x_{\min}, x_{\max}]$ اشتراک گرفته می‌شود: بخش سبز نوشته می‌شود و بخش خاکستری نه.

چرا این طراحی؟ برش جداگانه‌ی لبه‌ها برای رسم wireframe کافی است، اما داخل ستاره را محدود نمی‌کند. برای همین برش مرز و محدود کردن fill دو مرحله‌ی جدا در خط لوله هستند.

قرارداد مختصات و پیکسل

این قراردادها در همه‌ی توابع شما یکسان‌اند:

- مختصات هندسی y-up است، مثل اسلایدها: x به سمت راست و y به سمت بالا زیاد می‌شود و پیکسل $(0, 0)$ گوشه‌ی پایین و چپ صحنه است.
- پیکسل (i, j) همان نقطه‌ی (i, j) است؛ یعنی مرکز پیکسل روی نقطه‌ی صحیح می‌افتد. Bresenham با شماره‌ی پیکسل‌ها (عدد صحیح) کار می‌کند، ولی هندسه‌ی ستاره و خروجی برش می‌توانند اعشاری باشند.
- تصویر در حافظه یک آرایه‌ی NumPy با شکل $(H, W, 3)$ و نوع `uint8` است و سطر ۰ آن بالای تصویر قرار دارد. تبدیل y هندسی به سطر آرایه را `put_pixel` و `fill_span` در `support.py` انجام می‌دهند؛ در کد خودتان فرض نکنید که y در آرایه رو به پایین است.
- پنجره بسته است: مرز آن داخل حساب می‌شود، یعنی پیکسل‌های پنجره $x_{\min} \le i \le x_{\max}$ و $y_{\min} \le j \le y_{\max}$ هستند. اندازه‌ی صحنه 520×800 و پنجره `(120, 120, 679, 399)` است.
- در scan-line، پیکسل (i, j) وقتی پر می‌شود که نقطه‌ی (i, j) داخل چندضلعی (قاعده‌ی زوج–فرد) و داخل پنجره باشد. پیکسل‌هایی که نقطه‌شان دقیقاً روی لبه‌ی چندضلعی است در آزمون‌ها بررسی نمی‌شوند؛ فقط قاعده‌ی رأس‌ها را مثل Lecture ۴ رعایت کنید تا هیچ رأسی دو بار شمرده نشود. حالت‌های مرزی پنجره را `selfcheck.py` آزمون می‌کند.

اجرای برنامه و آزمون‌ها

بعد از نصب کتابخانه‌ها با `pip install numpy pillow pygame`، دستورهای زیر را در پوشه‌ی کد آغازین اجرا کنید. با `selfcheck.py` می‌توانید هر بخش را جداگانه آزمون کنید:

دستور	کار
`python screensaver.py`	اجرای صحنه‌ی زنده‌ی اصلی
`python selfcheck.py`	آزمون همه‌ی بخش‌های الزامی؛ تصویر تفاوت‌ها در پوشه‌ی `selfcheck_out` ذخیره می‌شود
`python selfcheck.py star`	آزمون فقط یک بخش؛ به جای `star` می‌توانید `bresenham` یا `clip` یا `fill` بنویسید
`python selfcheck.py aa`	آزمون بخش امتیازی
`python screensaver.py --export`	ساخت `frame_noaa.png` برای تحویل و `frame_aa.png` در صورت انجام بخش امتیازی؛ گزینه‌ی `--t` لحظه‌ی تصویر و گزینه‌ی `--crop X Y W H` ناحیه‌ی بزرگ‌نمایی را تعیین می‌کند
`python screensaver.py --bench`	فقط برای بخش امتیازی: زمان اجرا و خطا برای k های مختلف

کلیدهای پنجره‌ی زنده: `W` نمایش لبه‌ی سیمی، `Space` توقف، ← و → یک قدم عقب و جلو در حالت توقف، `S` ذخیره‌ی تصویر و `Esc` خروج؛ برای بخش امتیازی هم `A` برای روشن و خاموش کردن anti-aliasing و `K` برای تغییر k. اگر تابعی را هنوز ننوشته باشید، پیام `not implemented yet` می‌بینید و بقیه‌ی صحنه کار می‌کند؛ از `make_star_vertices` شروع کنید.

سمت راست شکل ۱ خروجی `--export` برای $t = 8.2$ ثانیه است: ستاره گوشه‌ی پایین‌راست پنجره را قطع کرده و قسمت بیرون پنجره دیده نمی‌شود. خروجی شما باید تقریباً پیکسل‌به‌پیکسل مثل آن باشد؛ چند پیکسل اختلاف روی لبه‌ها اشکالی ندارد.

زبان برنامه‌نویسی و ابزارها

این تمرین را فقط با Python ۳٫۱۰ یا بالاتر انجام دهید؛ چون صحنه و پنجره‌ی نمایش با Python نوشته شده‌اند. در `rasterize.py` فقط ابزارهای جدول زیر را می‌توانید استفاده کنید.

ابزارها و کتابخانه‌های مجاز	موارد غیرمجاز
• ماژول `math` • NumPy برای آرایه‌ها؛ عملیات روی کل آرایه فقط در بخش امتیازی (`fill_polygon_aa`) • توابع `put_pixel`، `fill_span`، `polygon_bbox`، `get_region` و `block_mean` از `support.py` • Pillow فقط برای بارگذاری و ذخیره‌ی تصویر در `support.py`	• هر تابع آماده‌ی رسم خط، برش یا پر کردن چندضلعی: `ImageDraw`، scikit-image، matplotlib، OpenCV، `pygame.draw` • کتابخانه‌های آماده‌ی polygon clipping و هندسه، مثل `shapely` • hard-code کردن ۱۰ رأس ستاره • تغییر هر فایلی جز `rasterize.py` • کپی کردن کد از منابع دیگر بدون ذکر منبع

قطعه کد آغازین

فایل `CG_HW02_Starter.zip` را از کورسز دانلود کنید. این فایل صحنه، توابع کمکی و آزمون‌ها را دارد. فقط `rasterize.py` مال شماست و توضیح کامل هر تابع در docstring آن آمده است. بقیه‌ی فایل‌ها، یعنی `support.py`، `scene.py`، `screensaver.py`، `selfcheck.py` و پوشه‌ی `golden/`، را تغییر ندهید؛ موقع نمره‌دهی نسخه‌ی خودمان از آن‌ها استفاده می‌شود. امضای توابع به این شکل است و نام تابع‌ها و پارامترها نباید عوض شود:

```
def make_star_vertices(cx, cy, r_outer, r_inner, theta)
def draw_line_bresenham(img, x0, y0, x1, y1, color)
def clip_line_cohen_sutherland(x0, y0, x1, y1, win)
def fill_polygon_scanline(img, polygon, color, win)
def fill_polygon_aa(img, polygon, color, win, k=4)     # bonus
```

تابع امتیازی مستقل است؛ ننوشتن آن صحنه‌ی اصلی را خراب نمی‌کند و جلوی گرفتن ۱۰۰ نمره‌ی کامل را هم نمی‌گیرد.

۳ بخش امتیازی اختیاری · حداکثر ۱۰ نمره

این بخش جزو ۱۰۰ نمره‌ی اصلی نیست و تا ۱۰ نمره‌ی اضافه دارد.

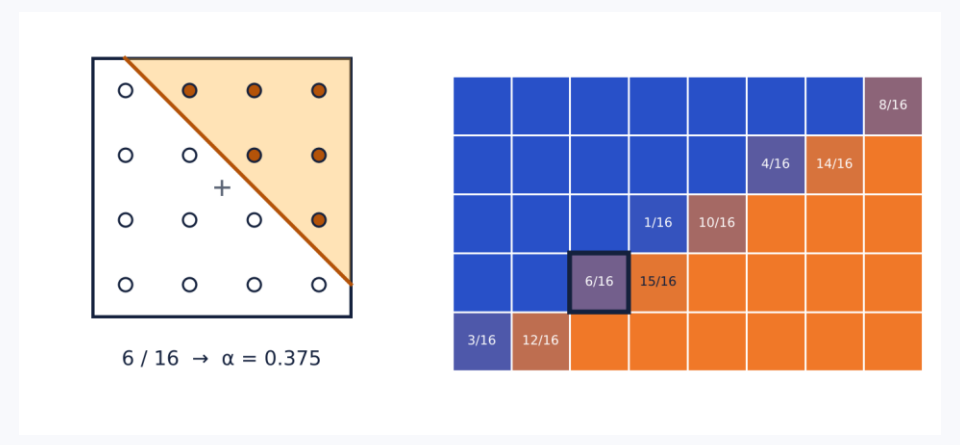
در خروجی بخش ۴، لبه‌ی ستاره پله‌ای (دندانه‌دار) است، چون هر پیکسل یا کاملاً داخل شکل حساب می‌شود یا کاملاً بیرون آن؛ ولی لبه‌ی واقعی از وسط پیکسل‌ها رد می‌شود. وقتی ستاره آرام می‌چرخد، این پله‌ها جابه‌جا می‌شوند و لبه

سوسو می‌زند. راه‌حل supersampling است: هر پیکسل را موقتاً به یک شبکه‌ی $k \times k$ از زیرنمونه‌ها تبدیل کنید. اگر از k^2 زیرنمونه‌ی یک پیکسل، n_{in} تا داخل قسمت دیده‌شده‌ی ستاره (داخل ستاره و داخل پنجره) بیفتد، پوشش آن پیکسل، یعنی coverage، برابر α است و رنگ نهایی را با همین α ترکیب می‌کنیم (همان ترکیب آلفای تمرین قبل):

$$\alpha = n_{\text{in}} \,/\, k^2$$

$$C_{\text{out}} = \alpha \cdot C_{\text{shape}} + (1 - \alpha) \cdot C_{\text{old}}$$

در اینجا C_{shape} رنگ ستاره و C_{old} رنگ قبلی پیکسل است. پس برای پیکسل‌های کاملاً داخل شکل α برابر ۱ و برای پیکسل‌های کاملاً بیرون آن برابر ۰ است؛ فقط پیکسل‌های لبه رنگ میانی می‌گیرند.



شکل ۵: چپ: ۱۶ نمونه‌ی یک پیکسل (علامت + نقطه‌ی (i, j) است) که ۶ نمونه‌ی آن داخل شکل است، یعنی $\alpha = 6/16 = 0.375$. راست: شکل نارنجی روی زمینه‌ی آبی؛ فقط پیکسل‌های لبه رنگ میانی دارند و عدد هر کدام تعداد نمونه‌های داخل شکل است.

برای نمونه‌برداری لازم نیست کد تازه‌ای بنویسید: همان `fill_polygon_scanline` خودتان وقتی روی یک mask بزرگ‌تر (به اندازه‌ی k برابر) اجرا شود، در هر پیکسل دقیقاً $k \times k$ نمونه می‌گیرد. تابع `fill_polygon_aa(img, polygon, color, win, k=4)` را با این مراحل بنویسید:

۱. با `polygon_bbox(polygon, w, h)` مستطیل محیطی (bounding box) ستاره را بگیرید و آن را به قسمتی از تصویر که پنجره می‌پوشاند محدود کنید؛ مختصات پنجره را هم مثل `polygon_bbox` با `floor(v + 0.5)` به پیکسل تبدیل کنید. فقط همین ناحیه ممکن است تغییر کند.

۲. یک mask صفر به اندازه‌ی $(w \cdot k) \times (h \cdot k)$ برای این ناحیه بسازید.

۳. رأس‌های ستاره و گوشه‌های پنجره را با همین نگاشت به مختصات mask ببرید (جمله‌ی $(k-1)/2$ باعث می‌شود زیرنمونه‌ها دور مرکز پیکسل قرار بگیرند):

$$(x, y) \to \big(k(x - b_{x0}) + (k-1)/2,\ k(y - b_{y0}) + (k-1)/2\big)$$

۴. `fill_polygon_scanline` خودتان را با ستاره‌ی بزرگ‌شده و پنجره‌ی بزرگ‌شده روی mask اجرا کنید و مقدار داخل را ۱ بگذارید.

۵. با `block_mean(mask, k)` از `support.py` میانگین هر بلوک $k \times k$ را بگیرید؛ نتیجه آرایه‌ای با مقدارهای ۰ تا ۱ است، یعنی α هر پیکسل.

۶. ناحیه‌ی مربوط را با `get_region(img, bx0, by0, bx1, by1)` بگیرید، با رابطه‌ی بالا با رنگ قبلی ترکیب کنید و در آخر به `uint8` برگردانید.

توجه: `fill_polygon_aa` باید از `fill_polygon_scanline` خودتان استفاده کند، نه از تابع آماده‌ی پر کردن. ترکیب را با اعداد اعشاری حساب کنید و فقط در آخر به `uint8` تبدیل کنید (گرد کردن یا حذف اعشار، هر دو قبول است). چون پنجره هم k برابر می‌شود، نمونه‌های بیرون پنجره شمرده نمی‌شوند و پیکسل‌های کنار مرز پنجره هم پوشش جزئی می‌گیرند.

مثال: رنگ شکل `(240, 120, 40)` و رنگ قبلی `(40, 80, 200)` است و پیکسلی $\alpha = 0.375$ دارد (۶ نمونه از ۱۶). کانال قرمز می‌شود $0.375 \times 240 + 0.625 \times 40 = 115$ و رنگ نهایی `(115, 95, 140)` است. همین مثال را با مستطیلی که گوشه‌هایش `(0.5, 0.5)` و `(1.25, 1.0)` است و $k = 4$ روی پیکسل $(1, 1)$ هم می‌توانید امتحان کنید؛ `selfcheck.py aa` همین حالت را می‌سنجد.

مقدارهای $k = 1, 2, 4, 8$ را امتحان کنید: هرچه k بزرگ‌تر باشد لبه نرم‌تر می‌شود ولی ساختن هر frame بیشتر طول می‌کشد. `python screensaver.py --bench` زمان هر frame و خطای هر k را نشان می‌دهد.

۴ گزارش

گزارش حداکثر ۳ صفحه و فقط PDF باشد. لازم نیست الگوریتم‌ها یا شبه‌کد اسلایدها را دوباره بنویسید؛ روی تصمیم‌های پیاده‌سازی، اتصال مراحل خط لوله و نتیجه‌ی اجرای خودتان تمرکز کنید. این موارد باید در گزارش باشد:

- یک تصویر از خروجی `--export` که در آن ستاره از مرز پنجره رد شده باشد
- توضیح کوتاه درباره‌ی اینکه ۱۰ رأس ستاره را چطور با تقارن پنج‌گانه ساختید و چرا hard-code کردن رأس‌ها با هدف تمرین جور نیست
- یک مثال واقعی از اجرای خودتان: مختصات یک لبه‌ی ستاره پیش از Cohen–Sutherland و بعد از آن
- مشکلات، بخش‌های ناقص، و فهرست منابع و ابزارهایی که از آن‌ها کمک گرفته‌اید
- پاسخ سه پرسش زیر، هر کدام در دو سه جمله

۱. چرا برش تک‌تک لبه‌های ستاره برای رسم wireframe کافی است، ولی برای اینکه ستاره روی قاب پر نشود باید هر span از scan-line را هم به پنجره محدود کنیم؟

۲. چرا در Cohen–Sutherland بعد از جابه‌جا شدن یکی از دو سر پاره‌خط، outcode آن دوباره محاسبه می‌شود؟

۳. چرا برای ساختن هر ۱۰ رأس ستاره، دانستن یک جفت رأس outer و inner و گام ۷۲ درجه کافی است؟ اگر ستاره‌ی هفت‌پر بود، کدام عددهای کد شما عوض می‌شد؟

اگر بخش امتیازی را انجام داده‌اید، یک جدول کوتاه برای $k = 1, 2, 4, 8$ شامل زمان اجرا (که از `--bench` می‌گیرید) و یک جمله درباره‌ی trade-off کیفیت و سرعت هم اضافه کنید.

۵ بارم‌بندی

نمره‌ی تمرین طبق جدول زیر محاسبه می‌شود.

بخش	نمره	معیار اصلی
تقارن و ساخت هندسه‌ی ستاره	۱۵	۱۰ رأس درست با تقارن پنج‌گانه
Bresenham	۲۰	همه‌ی جهت‌ها و محاسبات integer
Cohen–Sutherland	۲۵	برش صحیح لبه‌ها با پنجره و قرارداد y-up
scan-line و محدود کردن به پنجره	۳۰	چندضلعی مقعر و بدون رسم روی قاب
یکپارچه‌سازی، کیفیت کد و selfcheck	۵	خط لوله‌ی کامل و بدون تابع آماده
گزارش	۵	تصمیم‌ها، نتایج و پاسخ پرسش‌ها
جمع کل	**۱۰۰**	

بخش امتیازی (anti-aliasing) تا ۱۰ نمره‌ی اضافه دارد و نمره‌ی بخش اصلی همچنان از ۱۰۰ محاسبه می‌شود. در صورت استفاده از تابع آماده، نمره‌ی بخش مربوط صفر می‌شود.

۶ نحوه تحویل

قالب‌های زیر را دقیق رعایت کنید. رعایت نکردن این قوانین ممکن است باعث کسر نمره شود.

فایل‌های تحویلی

مورد	نام فایل	توضیح
فایل فشرده	`CG_HW02_StudentID_LastName.zip`	همه‌ی فایل‌ها در یک پوشه
کد	`rasterize.py`	فقط همین یک فایل کد
گزارش	`Report_HW02_StudentID.pdf`	حداکثر ۳ صفحه، فقط PDF
تصاویر خروجی	`frame_noaa.png` `frame_aa.png`	خروجی `--export` شما، به اندازه‌ی ۸۰۰ در ۵۲۰؛ فایل دوم فقط در صورت انجام بخش امتیازی

- فایل‌های `scene.py`، `support.py`، `screensaver.py`، `selfcheck.py` و پوشه‌ی `golden/` را داخل فایل فشرده نگذارید.
- در نام فایل فشرده، `02` شماره‌ی تمرین، `StudentID` شماره‌ی دانشجویی (فقط رقم) و `LastName` نام خانوادگی شما به انگلیسی (فقط حروف) است.
- در نام فایل‌ها فقط حروف انگلیسی، عدد و خط زیر (`_`) به کار ببرید؛ فاصله و حروف فارسی مجاز نیست. فقط فرمت `zip` پذیرفته می‌شود و فایل‌های `rar` و `7z` بررسی نمی‌شوند.

ساختار فایل فشرده

با باز کردن فایل `zip` باید پوشه‌ای هم‌نام با آن و با ساختار زیر وجود داشته باشد:

```
CG_HW02_40112345_Ahmadi/
├── rasterize.py
├── Report_HW02_40112345.pdf
├── frame_noaa.png
└── frame_aa.png          # bonus only
```

فایل‌های اضافه، مثل `__pycache__`، را پاک کنید.

بارگذاری در کورسز

۱. فایل `zip` را در سامانه‌ی کورسز بارگذاری کنید.

۲. بعد از بارگذاری، فایل را دانلود کنید و مطمئن شوید درست باز می‌شود. در صورت نیاز می‌توانید نسخه‌ی اصلاح‌شده را دوباره بارگذاری کنید و نسخه‌ی قبلی را حذف کنید؛ فقط آخرین نسخه بررسی می‌شود.

چک‌لیست پیش از ارسال

- ☐ `python selfcheck.py` برای همه‌ی بخش‌های الزامی بدون خطا اجرا می‌شود.
- ☐ ستاره از تقارن پنج‌گانه ساخته شده و ۱۰ رأس hard-code نشده است.
- ☐ وقتی ستاره وارد ناحیه‌ی قاب می‌شود، هیچ قسمتی از لبه یا رنگ آن روی قاب دیده نمی‌شود.
- ☐ مختصات همه‌جا y-up است و تبدیل به سطر تصویر فقط با `put_pixel` و `fill_span` انجام می‌شود.
- ☐ در کد از تابع آماده‌ی رسم، برش یا پر کردن استفاده نشده است.
- ☐ نام فایل‌ها و ساختار `zip` مطابق جدول بالا است و فایل بارگذاری‌شده یک بار دوباره دانلود و باز شده است.

موفق باشید · تیم درس گرافیک کامپیوتری